

Corrigé — Baccalauréat NSI

Amérique du Nord 2026

Exercice 1 — Puissance 4, POO, récursivité

Partie A — Grille de jeu et score

1. Méthode `__init__` de la classe `Grille`

```
class Grille:
    def __init__(self):
        self.grille = [[0 for colonne in range(7)] for ligne in range(6)]
```

2. Méthode `joue`

```
def joue(self, colonne, joueur):
    ligne = 5
    while ligne != -1 and self.grille[ligne][colonne] != 0:
        ligne = ligne - 1

    if ligne != -1:
        self.grille[ligne][colonne] = joueur
        return True
    else:
        return False
```

3. Création de la grille `jeu1`

```
jeu1 = Grille()
jeu1.joue(3, 2)
jeu1.joue(2, 1)
jeu1.joue(3, 1)
```

4. Score de la grille jeu1

La grille contient :

- un pion du joueur 2 en (5, 3) ;
- un pion du joueur 1 en (5, 2) ;
- un pion du joueur 1 en (4, 3).

```
score = valeur_case(5, 3) - valeur_case(5, 2) - valeur_case(4, 3)
score = 7 - 5 - 10
score = -8
```

Score associé à jeu1 : -8.

5. Méthode score

```
def score(self):
    total = 0
    for ligne in range(6):
        for colonne in range(7):
            if self.grille[ligne][colonne] == 2:
                total += valeur_case(ligne, colonne)
            elif self.grille[ligne][colonne] == 1:
                total -= valeur_case(ligne, colonne)
    return total
```

Partie B — Algorithme min-max

6. Classe Noeud

```
class Noeud:
    def __init__(self, colonne):
        self.colonne = colonne
        self.score = 0
        self.suivants = []
```

7. Méthode colonne_score_min

```
def colonne_score_min(self):
    meilleur = self.suivants[0]

    for noeud in self.suivants:
        if noeud.score < meilleur.score:
            meilleur = noeud

    return (meilleur.colonne, meilleur.score)
```

8. Méthode `calcule_score`

```
def calcule_score(self, niveau, joueur, grille):
    g = grille.gagnant()

    if g == 1:
        self.score = -(100 + 10 * (niveau_max - niveau))

    elif g == 2:
        self.score = 100 + 10 * (niveau_max - niveau)

    elif niveau == niveau_max:
        self.score = grille.score()

    else:
        for colonne in range(7):
            grille2 = grille.copie_grille()

            if grille2.joue(colonne, joueur):
                nouveau_noeud = Noeud(colonne)
                self.suivants.append(nouveau_noeud)

                if joueur == 1:
                    joueur_suivant = 2
                else:
                    joueur_suivant = 1

                nouveau_noeud.calcule_score(
                    niveau + 1,
                    joueur_suivant,
                    grille2
                )

        if joueur == 1:
            self.score = self.colonne_score_min()[1]
        else:
            self.score = self.colonne_score_max()[1]
```

9. Pourquoi ne pas explorer toutes les parties ?

Il n'est pas réaliste d'explorer toutes les parties avec `niveau_max = 42`, car à chaque coup il peut y avoir jusqu'à 7 possibilités.

Nombre de chemins possibles, dans le pire cas : 7^{42}

Ce nombre est gigantesque : l'algorithme demanderait beaucoup trop de temps de calcul et de mémoire.

Partie C — Choix du meilleur coup

10. Fonction choisit_coup

```
def choisit_coup(grille, joueur):
    racine = Noeud(-1)

    for colonne in range(7):
        grille2 = grille.copie_grille()

        if grille2.joue(colonne, joueur):
            noeud = Noeud(colonne)
            racine.suivants.append(noeud)

            if joueur == 1:
                joueur_suivant = 2
            else:
                joueur_suivant = 1

            noeud.calculer_score(1, joueur_suivant, grille2)

    if joueur == 1:
        return racine.colonne_score_min()[0]
    else:
        return racine.colonne_score_max()[0]
```

Exercice 2 — Réseaux, files, POO

1. Réseau Administration

Le réseau Administration est 10.42.0.80/29.

Nombre total d'adresses : $2^{(32 - 29)} = 8$

Adresses attribuables : $8 - 2 = 6$

Les adresses attribuables vont de 10.42.0.81 à 10.42.0.86.

Une adresse possible pour le routeur E est donc 10.42.0.81.

2. Réseau de l'adresse 10.42.0.70

Le réseau Gaming Online est 10.42.0.64/28.

Adresses du réseau : 10.42.0.64 à 10.42.0.79

Donc 10.42.0.70 appartient au réseau **Gaming Online**.

3. Adresses des routeurs sur les réseaux en /30

Réseau 10.42.0.16/30 :

10.42.0.16 : adresse réseau

10.42.0.17 : routeur C

10.42.0.18 : routeur F

10.42.0.19 : adresse de diffusion

Réseau 10.42.0.12/30 :

10.42.0.12 : adresse réseau

10.42.0.13 : routeur C

10.42.0.14 : routeur D

10.42.0.15 : adresse de diffusion

4. Table RIP possible du routeur C

Réseau	Passerelle	Nombre de sauts
Gaming Online	connecté	0
Gaming VR	10.42.0.18	1
DMZ	10.42.0.1	1
Internet	10.42.0.1	2
Application	10.42.0.14	1
Administration	10.42.0.1	2
SGBD	10.42.0.14	2

5. Coûts OSPF

Formule :

coût = 10^{10} / débit

Débit	Débit en bit/s	Coût
10 Gb/s	10^{10}	1
1 Gb/s	10^9	10
100 Mb/s	10^8	100

6. Chemin OSPF de A vers Application

On cherche le chemin de coût minimal de A vers le routeur D.

A → B → E → D

Coût :

A-B : 1
B-E : 10
E-D : 1
Total : 12

7. Encapsulation TCP/IP

Les données d'un segment TCP sont contenues dans un paquet IP.

On dit que TCP est encapsulé dans IP.

8. File ou pile ?

Une file fonctionne en FIFO : *First In, First Out*.

Le premier paquet arrivé est le premier transmis. C'est plus équitable qu'une pile, qui ferait passer les paquets les plus récents avant les plus anciens.

9. Initialisation de Routeur_DROP_TAIL

```
class Routeur_DROP_TAIL:
    def __init__(self, t_max):
        self.f = cree_file()
        self.t_max = t_max
        self.t = 0
```

10. Méthode recoit de Routeur_DROP_TAIL

```
def recoit(self, p):
    if self.t < self.t_max:
        enfile(self.f, p)
        self.t = self.t + 1
        return True
    return False
```

11. Méthode recoit de Routeur_ALEA

```
def recoit(self, p):
    if self.t < self.t_min:
        enfile(self.f, p)
        self.t = self.t + 1
        return True

    elif self.t_min <= self.t < self.t_max:
        if self.tirage_au_sort():
            enfile(self.f, p)
            self.t = self.t + 1
            return True
        return False

    return False
```

Exercice 3 — SQL, récursivité, programmation dynamique

Partie A — SQL

1. Clé primaire de la table immeuble

Le numéro d'un immeuble dans une rue ne suffit pas à identifier un immeuble de manière unique.

Deux rues différentes peuvent par exemple contenir chacune un immeuble numéro 13.

2. Immeubles de la rue 'la mer'

```
SELECT id_immeuble
FROM immeuble
WHERE rue_immeuble = 'la mer'
ORDER BY id_immeuble;
```

3. Appartements de l'immeuble 16 au moins au 5e étage

```
SELECT id_appart
FROM appartement
WHERE id_immeuble = 16
AND etage_appart >= 5;
```

4. Intégrité référentielle

La table appartement contient une clé étrangère id_immeuble qui référence la table immeuble.

Si on supprime l'immeuble 16 alors que des appartements y sont encore associés, certains appartements pointeront vers un immeuble inexistant.

5. Insertion d'un nouvel immeuble

```
INSERT INTO immeuble(id_immeuble, nb_etage_immeuble, numero_immeuble, rue_immeuble)
VALUES (140, 6, 13, 'Turing');
```

6. Doubler le prix de l'appartement 603

```
UPDATE appartement
SET prix_appart = 2 * prix_appart
WHERE id_appart = 603;
```


7. Prix maximal d'un appartement dans un immeuble de la rue 'la mer'

```
SELECT MAX(prix_appart)
FROM appartement
JOIN immeuble ON appartement.id_immeuble = immeuble.id_immeuble
WHERE rue_immeuble = 'la mer';
```

Partie B — Plus longue sous-séquence strictement croissante

8. Sous-séquences strictement croissantes de longueur 2 de L2 = [3, 1, 8, 2, 5]

```
[3, 8]
[3, 5]
[1, 8]
[1, 2]
[1, 5]
[2, 5]
```

9. Plus longue sous-séquence strictement croissante de L2

```
[1, 2, 5]
```

Elle est de longueur 3.

10. Fonction est_strict_croissante

```
def est_strict_croissante(seq):
    for i in range(1, len(seq)):
        if seq[i] <= seq[i - 1]:
            return False
    return True
```

11. Fonction récursive llsc_fin

```
def llsc_fin(tab, i):
    if i == 0:
        return 1

    max_len = 1

    for j in range(i):
        if tab[j] < tab[i]:
            max_len = max(max_len, llsc_fin(tab, j) + 1)

    return max_len
```

12. Fonction dynamique llsc_dyn

```
def llsc_dyn(tab):  
    n = len(tab)  
    dyn = [1] * n  
  
    for i in range(1, n):  
        for j in range(i):  
            if tab[j] < tab[i]:  
                dyn[i] = max(dyn[i], dyn[j] + 1)  
  
    return max(dyn)
```

13. Avantage de la programmation dynamique

L'implémentation dynamique évite de recalculer plusieurs fois les mêmes valeurs.

C'est une **programmation dynamique ascendante** : on calcule d'abord les solutions des sous-problèmes les plus simples, puis on les utilise pour construire les solutions suivantes.

Ici, `dyn[i]` est calculé à partir des valeurs déjà connues `dyn[j]` pour $j < i$.